Retrieval-Augmented Generation (RAG)

Topics on NLP & LLM
- ADDI Laboratory -

Rarasmaya Indraswari
(WA) 081331818454
raras@its.ac.id

RAG

LLM

- Model bahasa besar dilatih dari data masif untuk next-token prediction → menghasilkan teks, menjawab pertanyaan, summarization, reasoning.
- Contoh arsitektur: GPT, Llama, PaLM, Mistral, T5.
- Kemampuan saat ini:
  - Text generation (chat, essay, explanative output)
  - Reasoning terarah (chain-of-thought, agent)
  - Code generation
  - Instruction following
  - Multimodal (teks + gambar + suara)

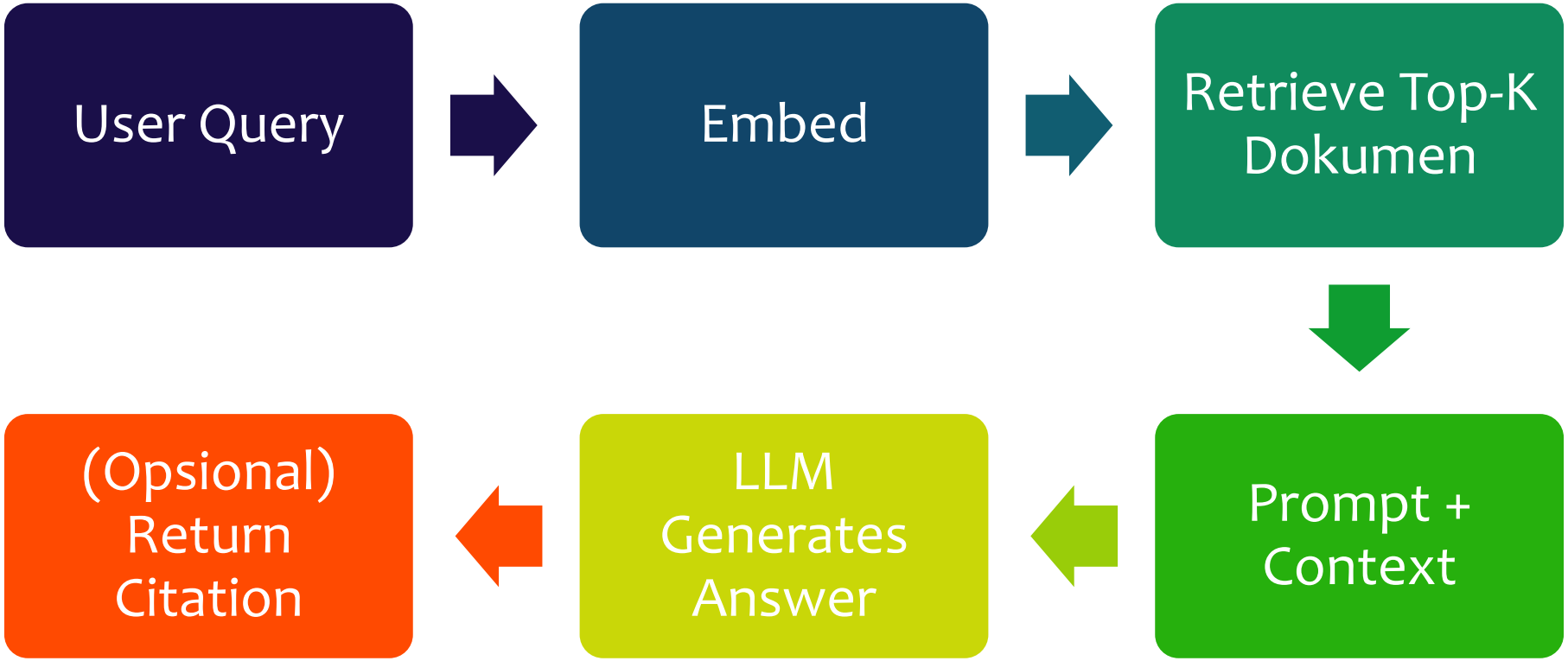
Permasalahan LLM

Keterbatasan	Penjelasan
Hallucination	Model mengarang fakta karena forecasting token → bukan retrieval knowledge.
Static knowledge	Model tidak tahu hal terbaru setelah tanggal training.
Tidak transparan (black box)	Sulit verifikasi sumber pengetahuan.
Tidak bisa grounded workflow	Perlu data yang dapat divalidasi, bukan sekadar prediksi token.

RAG

- RAG = Pipeline yang menggabungkan Information Retrieval + LLM Generation.
  - Instead of relying on parametric memory, model di-augment dengan dokumen dari luar.
  - Pendekatan non-parametric knowledge injection.
- Sehingga LLM menghasilkan jawaban berdasarkan informasi spesifik yang diambil dari corpus, bukan dari hafalan internal.

Arsitektur Dasar



Komponen RAG

- Corpus / Knowledge Source
  - dokumen: PDF, HTML, CSV, DB, intranet, dsb.
- Indexer / Vector Store
  - ubah dokumen → embedding → simpan di vector DB (FAISS, Milvus, Pinecone, Weaviate)
- Retriever
  - kembalikan top-k dokumen/paragraph berdasar cosine / ann search (dpr/dense retriever atau sparse seperti BM25)
- Ranker (opsional)
  - rerank hasil retrieval (cross-encoder).
- Generator (LLM)
  - concat prompt + retrieved context → model generatif (T5, Flan, Llama, GPT).
- Post-processing / Citation
  - ringkasan jawaban + sertakan referensi/passage.

Kelebihan RAG

Aspek	Keuntungan
Akurasi faktual	Mengurangi hallucination karena jawaban berdasarkan dokumen
Update mudah	Tinggal perbarui index, bukan retraining model
Interpretabilitas	Bisa menampilkan sumber/citation
Lebih murah dari fine-tuning	Tidak perlu backprop model besar

Keterbatasan RAG

Tantangan	Konsekuensi
Retrieval salah → jawaban salah	"Garbage in, garbage out"
Latensi meningkat	Query embedding + search + generation
Butuh manajemen corpus	Data cleaning, security, chunking

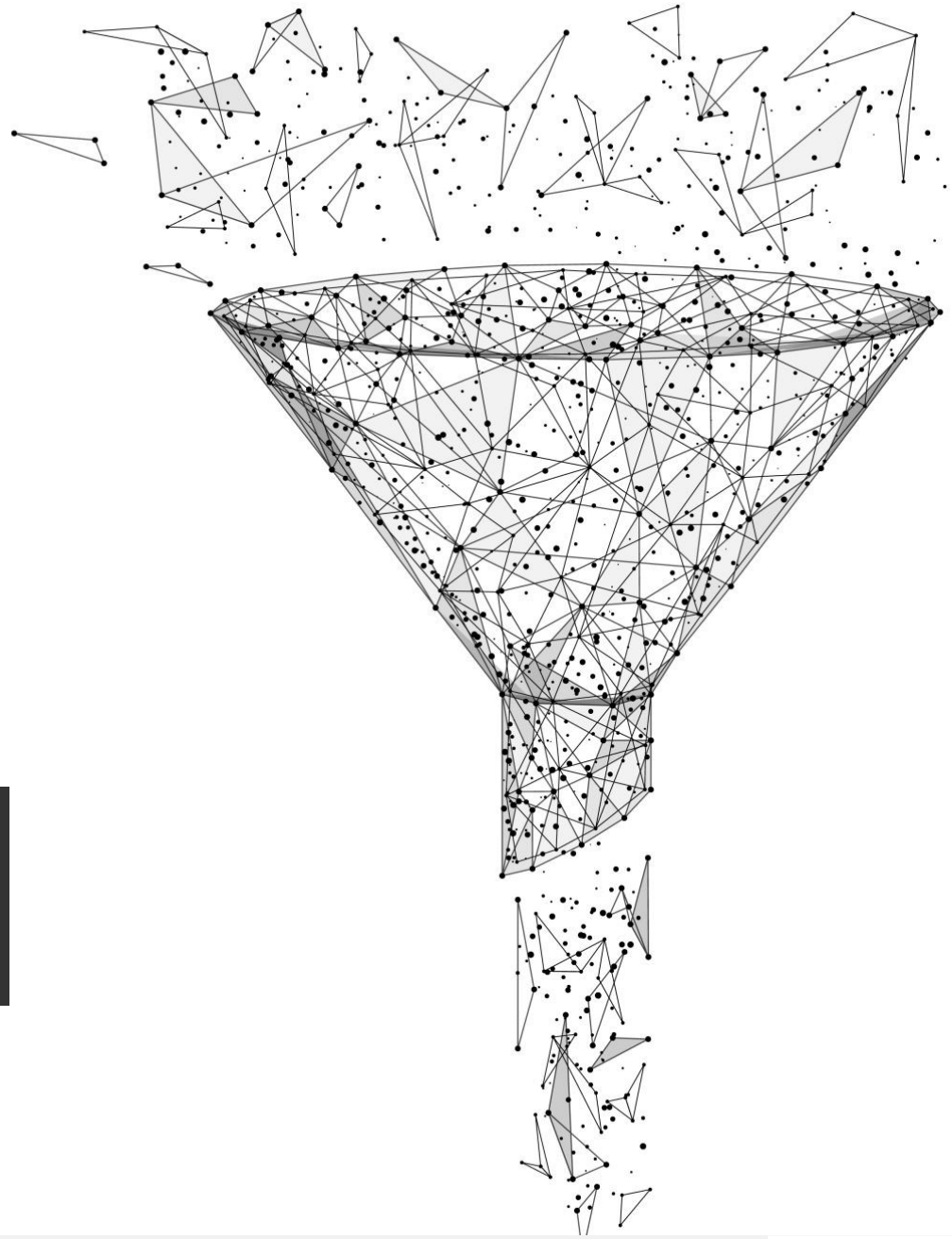
Variasi Metode RAG

Tipe RAG	Ciri	Kapan digunakan
RAG klasik	retrieve → prompt → generate	chatbot pengetahuan, tanya jawab dokumen
RAG dengan re-ranking	retriever + cross-encoder ranker	dataset besar, noise tinggi
RAG as Agents	LLM bisa memilih dokumen/aksi	workflow multi-step
RAG untuk data real-time	streaming retrieval + caching	berita, pricing, sensor IoT
Hybrid Search (BM25 + Dense)	gabungkan retriever semantic & lexical	query campuran natural language + keyword

Contoh Aplikasi RAG

Domain	Contoh Implementasi
Kampus / Akademik	Chatbot tanya jawab peraturan akademik, SIAKAD, jurnal
Legal	Tanya jawab berdasarkan dokumen kontrak
Medis	Q&A berdasarkan guideline klinis
Customer Support	Bot dokumen SOP internal
Knowledge Base Perusahaan	Chat internal berbasis dokumen per divisi

LangChain Framework



Latar Belakang

- Membangun RAG tanpa framework:

Loader → chunking → embedding → vector store → retriever → LLM → formatting → memory → evaluation → API serving

- Perlu mengintegrasikan banyak library secara manual → rumit dan tidak modular
- LangChain: framework python untuk membangun aplikasi LLM modular, memudahkan prototyping RAG

Fitur LangChain

Fitur	Fungsi
Document Loaders	impor PDF, web, SQL, GDrive
Text Splitters	chunking otomatis
Embeddings & VectorStores	FAISS, Pinecone, Milvus
Chains	pipeline modular
Agents & Tool-Calling	reasoning multi langkah
Memory	percakapan stateful

Keterbatasan LangChain

- Masalah saat aplikasi makin kompleks:
  - Butuh state yang persist & dapat di-*resume*
  - Butuh agent multi-node
  - Butuh workflow branching
  - Butuh audit trail langkah reasoning
  - Butuh concurrency & long-running processes

- LangChain linear pipeline tidak cukup untuk sistem agen kompleks.

LangGraph

- Framework berbasis graph stateful untuk membangun sistem agen multi-step.
- Karakteristik:

Fitur	Penjelasan
Graph-based workflow	node mewakili fungsi/agent
Bisa looping & branching	tidak seperti chain linear
Stateful	context bertahan antar langkah
Cocok untuk multi agent	misal “planner agent” + “retriever agent”

Contoh Proses LangGraph

```
Query → Planner Agent → Retriever Node → LLM Node → Evaluator Node → Output
          ↑---------------------------------------- Loop reasoning
```

Use-Case LangGraph

- Research assistant multi langkah
- Sistem hukum yang memverifikasi dokumen
- Autonomous data pipeline
- Multi-agent scientific reasoning

RAG Lokal

- Download & Install Ollama for Windows
- Buka window cmd (command prompt
- Untuk download model gunakan perintah “pull”, seperti : ollama pull llama3.1
- Ollama serve → http://localhost:11434

Final Project

Tugas 1

- Sifat : Individual
- Task : Membuat LLM RAG-based chatbot system
- Studi kasus dan dataset bebas (kalau QA minimal 100)
- Boleh menggunakan API atau lokal
- Metrik evaluasi : minimal 2 antara BERTScore, BLEU, ROUGE, precision@k, manual evaluation, dsb.
- Tugas tambahan : Bandingkan hasil antara dengan RAG dan tanpa RAG minimal untuk 5 query
- Yang dikumpulkan: Program, dataset, PPT (latar belakang, problem, dataset, metodologi, hasil dan discussion), video rekaman PPT + demo program (max 15 menit)
- Deadline : 14 Desember 23.59 WIB

Tugas 2

- Sifat : Individual
- Task : Versi advanced dari Tugas 1
- Tambahkan (pilih minimal 1) :
  - Fine tuning (full atau PEFT), bandingkan hasilnya – minimal mencoba 1 jenis
  - Menggunakan knowledge distillation, bandingkan hasilnya
  - XAI, berikan penjelasan + analisis
  - Reranking (seperti cross-encoder), bandingkan hasilnya
- Yang dikumpulkan: Program, dataset, PPT (latar belakang, problem, dataset, metodologi, hasil dan discussion), video rekaman PPT + demo program (max 15 menit)
- Deadline: 21 Desember 23.59 WIB

Thank You